```
vif["VIF"] = [
  variance_inflation_factor(x.values, i)
  for i in range(x.shape[1])
]
```

This code calculates the Variance Inflation Factor (VIF) for every independent variable in the dataset and stores the results in the VIF column of the DataFrame.

1. x.values

- Converts the Pandas DataFrame x into a NumPy array.
- variance_inflation_factor() uses this numerical array to calculate VIF.

For example, if x contains five columns (ssc_p, hsc_p, degree_p, etest_p, and mba_p), it passes all five columns as a numerical array.

2. x.shape[1]

```
x.shape[1]
```

- x.shape returns the number of rows and columns.
- x.shape[0] gives the number of rows.
- x.shape[1] gives the number of columns.

For example:

```
x.shape
# (100, 5)
```

Here, there are 100 rows and 5 columns.

```
x.shape[1]  # 5
```

3. range(x.shape[1])

```
range(x.shape[1])
```

If there are five columns, this generates the column indices:

0, 1, 2, 3, 4

Each index represents one independent variable.

4. variance_inflation_factor(x.values, i)

```
variance_inflation_factor(x.values, i)
```

This function calculates the VIF for the column at index i, using the other columns as predictors.

For example:

- i = 0 → calculates VIF for ssc_p.
- i = 1 → calculates VIF for hsc_p.
- i = 2 → calculates VIF for degree_p.
- i = 3 → calculates VIF for etest_p.
- i = 4 → calculates VIF for mba_p.

It repeats this calculation for every column.

5. List comprehension

```
[
  variance_inflation_factor(x.values, i)
  for i in range(x.shape[1])
]
```

This is called a list comprehension. It combines the loop and calculation into a single line.

```
for i in range(x.shape[1]):
  print(i)
```

x has five columns.

```
x.shape
# (100, 5)
```

This means there are 100 rows and 5 columns.

- x.shape[0] → 100, the number of rows.
- x.shape[1] → 5, the number of columns.

```
range(x.shape[1])
range(5)
```

Python generates the numbers 0, 1, 2, 3, 4.

Notice that range(5) stops before 5.

How the loop executes

Assume the DataFrame has five independent variables.

0

i = {item.i}

Calculates the VIF for {item.name}

1

i = {item.i}

Calculates the VIF for {item.name}

2

i = {item.i}

Calculates the VIF for {item.name}

3

i = {item.i}

Calculates the VIF for {item.name}

4

i = {item.i}

Calculates the VIF for {item.name}

The loop repeats five times because there are five columns.

During each iteration, i holds a different column index.

Iteration	**Value of i**	**Variable being evaluated**
1	0	ssc_p
2	1	hsc_p
3	2	degree_p
4	3	etest_p

It is equivalent to writing:

```
vif_values = []

for i in range(x.shape[1]):

   value = variance_inflation_factor(x.values, i)

   vif_values.append(value)

vif["VIF"] = vif_values
```

Both approaches calculate the same VIF values.

6. vif["VIF"] = ...

Finally, this assigns the calculated VIF values to a new column named VIF in the vif DataFrame.

For example, the resulting DataFrame might look like this:

Variable	**VIF**
ssc_p	2.1
hsc_p	3.4
degree_p	1.8
etest_p	1.2
mba_p	2.7

These values are illustrative, not actual results from your dataset.